



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1: Especificacion de Tads

Tad AirMiles

September 13, 2026

Algoritmos y Estructuras de Datos

Grupo: Arraydecuatroposiciones

Integrante	LU	Correo electrónico
Javier M. Ramos	288/20	ramosjavier276@gmail.com
Nancy Ramos	1206/24	Ramosnancydaniela@outlook.com
Elian Carreño	1420/23	elianuba2004@gmail.com
Leandro Amodey	618/24	leandroamodey@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

<http://www.exactas.uba.ar>

1 Introducción

El presente trabajo practico es una especificacion formal para un sistema de recompensas basado en millas, como ocurre en muchas aerolineas reales.

**** VER CUANTO DETALLE DE NUESTRA INTERPRETACION PONEMOS EN LA INTRO ****

2 Renombres de Tipos y Estructuras

enum Categoría **ES** {Bronce, Plata,Oro, Platino }

Transacciones **ES** Struct $\langle UsuarioOrigen : \mathbb{Z},$
 $UsuarioDestino : \mathbb{Z}, TipoDeTransaccion : \mathbb{Z}, Millas : \mathbb{R},$
 $NombrePromo : \text{string}, FactorPromo : \mathbb{R} \rangle$

Tipo de transacciones es algo que yo pongo a mano creo, ejemplo 0: Transaccion por referidor, 1: Transferencia, 2: Canjeo de puntos

3 Especificación del TAD AirMiles

TAD Airmiles {

Decidimos dejar en usuarios la categoria por mas que se pueda calcular por fuera, historial de transacciones cambio a un tipo Struct, y las promociones tambien cambiaron a un diccionario que adentro tiene un struct

obs Usuarios : dict $\langle id : \mathbb{Z}, CategoriaActual : Categoria \rangle$

La convencion utilizada para transacciones es:

1) Si se hace un vuelo, el TipoDeTransaccion es 0 y el usuario de destino es -1

2) Si se hace una transferencia el TipoDeTransaccion es 1

3) Si se hace un canjeo el TipoDeTransaccion es 2 y el usuario de destino es -1, se modifica unicamente el usuario de origen

obs Historial : seq $\langle Transacciones \rangle$

En caso de las promociones, si la promocion no tiene un tope, por convencion tomamos millasRestantes = -1

obs Promociones : dict {

NombrePromocion: string,

$\langle FactorDePromocion : \mathbb{R}, topeDePromocion : \mathbb{R}, millasRestantes : \mathbb{R} \rangle$

}

proc crearSistema() : AirMiles {

requiere { True }

asegura {

$|res.Usuarios| = 0 \wedge$

$|res.Historial| = 0 \wedge$

$res.promociones = setKey(dict\langle, \rangle, "NoPromo", \langle 1, 0, -1 \rangle)$

}

}

proc registrarUsuario(inout AM : Airmiles, in id : $\mathbb{Z}$) {

Actualiza el diccionario de usuarios con nuevo usuario

requiere { $AM = AM0 \wedge id \notin claves(AM.Usuarios) \wedge id > 0$ }

```

    asegura {
      AM.Usuarios = SetKey(AM0.Usuarios, id, Bronce) ∧
      AM.Historial = AM0.Historial ∧
      AM.Promociones = AM0.Promociones
    }
  }
}

proc registrarUsuarioConReferidor(inout AM : Airmiles,
in idReferidor : ℤ, in idUsuario : ℤ) {
  requiere { AM0 = AM ∧ idReferidor ∈ claves(AM.Usuarios) ∧ idUsuario ∉
claves(AM.Usuarios) ∧ idUsuario > 0 }
  asegura {
    AM.Usuarios = SetKey(AM0.Usuarios, idUsuario, Bronce) ∧
    EsNuevaTransaccion(AM, AM0, idUsuario, -1, 0,
500, "NoPromo", 1) ∧
    AM.Promociones = AM0.Promociones
  }
}

Acumula millas al usuario si la categoria del mismo no es platino, en caso de
que el nombre de la promo no alcance para completar el viaje del usuario, se
crean dos transacciones, una con las millas restantes de la promo y otra con
"NoPromo" para completar el viaje.

proc AcumularMillas(inout AM : Airmiles, in idUsuario : ℤ, in distancia : ℝ, in nombrePromo :
string,) {
  requiere { AM = AM0 ∧ idUsuario ∈ claves(AM.Usuarios) ∧ nombrePromo ∈
claves(AM.Promociones) ∧ distancia > 0 }
  asegura {
    Si el tope de las millas de la promo supera la distancia se hace una sola transaccion
    AM0.Promociones[nombrePromo].millasRestantes ≥
    MillasCalculadas(distancia, DarFactorCategoria(AM.Historial, idUsuario),
    AM.Promociones[nombrePromo].factorDePromocion) →L

    AGREGAR EL CASO DE BAJAR EL TOPE CUANDO SE CONSUME LA PROMOCION
    EsNuevaTransaccion(AM, AM0, idUsuario, -1, 0,
    MillasCalculadas(distancia, DarFactorCategoria(AM.Historial, idUsuario),
    AM.Promociones[nombrePromo].factorDePromocion), distancia,
    nombrePromo, AM0.Promociones[nombrePromo].FactorDePromocion) ∧

    AM.Promociones = setKey(AMO.Promociones, nombrePromo,
    AM0.Promociones[nombrePromo].millasRestantes - MillasCalculadas
    (distancia, DarFactorCategoria(AM.Historial, idUsuario),
    AM.Promociones[nombrePromo].factorDePromocion) ∨

    AM0.Promociones[nombrePromo].millasRestantes <
    MillasCalculadas(distancia, DarFactorCategoria(AM.Historial, idUsuario),
    AM.Promociones[nombrePromo].factorDePromocion) →

    Se hace 1 transaccion con las millas restantes de promo
    EsNuevaTransaccion(AM, AM0, idUsuario, -1, 1,
    AM0.Promociones[nombrePromo].millasRestantes, distancia,

```

```

nombrePromo, AM0.Promociones[nombrePromo].FactorDePromocion)  $\wedge_L$ 

Se hace otra transaccion con noPromo para completar
EsNuevaTransaccion(AM, AM0, idUsuario, -1, 1,
millasACompletar(AM0.Promociones, nombrePromo, distancia,
DarFactorCategoria(AM0.Historial, idUsuario),
distancia, "NoPromo", 1)  $\wedge_L$ 
Se puede indefinir si no le pasamos un usuario que este en el sistema
SubioCategoria(idUsuario, AM.Historial, AM0.Historial)  $\rightarrow_L$ 
AM.Usuarios = SetKey(AM0.Usuarios, idUsuario,
DarCategoria(AM.Historial, idUsuario))  $\wedge$ 
Despues de completar el viaje se verifica si la promocion termino
SeConsumioLaPromocion(nombrePromo, AM, AM0)
}
}

```

```

proc CanjearMillas(inout AM : AirMiles, in MillasCanjeadas :  $\mathbb{R}$ , in IdUsuario :  $\mathbb{Z}$ )
{
  requiere {
    AM = AM0  $\wedge_L$  MillasCanjeadas > 0  $\wedge$  IdUsuario  $\in$  claves(AM.Usuarios)
  }
  asegura {
    IfThenElse(MillasDisponibles(IdUsuario, AM0.Historial)  $\geq$  MillasCanjeadas,
    , EsNuevaTransaccion(AM, AM0, IdUsuario, -1, 2,
    MillasCanjeadas, 0, NoPromo, 0), AM.Historial = AM0.Historial)
     $\wedge$  AM.Usuarios = AM0.Usuarios
     $\wedge$  AM.Promociones = AM0.Promociones
  }
}

```

```

proc TransferirMillas(inout AM : Airmiles,
in idUsuarioPagador :  $\mathbb{Z}$ , in idUsuarioBeneficiario :  $\mathbb{Z}$ , in CantMillasTrans :  $\mathbb{R}$ ) {
  requiere { AM0 = AM  $\wedge$  idUsuarioPagador  $\in$  claves(AM.usuarios)
 $\wedge$  idUsuarioBeneficiario  $\in$  claves(AM.usuarios)
 $\wedge$  idUsuarioPagador  $\neq$  idUsuarioBeneficiario
 $\wedge$  MillasDisponibles(idUsuarioPagador, AM.Historial)  $\geq$  CantMillasTrans }
  asegura {
    EsNuevaTransaccion(AM, AM0, idUsuarioPagador, idUsuarioBeneficiario, 1, CantMillasTrans)
 $\wedge$  AM.Promociones = AM0.Promociones
 $\wedge$  AM.Usuarios = AM0.Usuarios  $\wedge$  SubioCategoria(idUsuarioBeneficiario)
  }
  Revisado
}

```

```

proc ConsultarSaldo(in AM : AirMiles, in IdUsuario :  $\mathbb{Z}$ ) :  $\mathbb{Z}$  {
  requiere {
    IdUsuario  $\in$  claves(AM.Usuarios)
  }
  asegura {
    res = MillasDisponibles(IdUsuario, AM.Historial)
  }
} El auxiliar MillasDisponibles es el que esta en CanjearMillas. Ver despues si reusamos ese para todo u algun otro, pero hay que unificar.

```

```

proc ConsultarCategoria(in AM : Airmiles, in idUsuario :  $\mathbb{Z}$ ,) : categoria {
  requiere { idUsuario  $\in$  claves(AM.usuarios) }
  asegura { res = AM.Usuarios[idUsuario] }
}

```

```

proc RankingMillas(in AM : Airmillas, in n :  $\mathbb{Z}$ ) : seq< $\mathbb{Z}$ > {
  requiere {
    n > 0
  }
  asegura {
    (|AM.Usuarios| > n  $\wedge_L$  |res| = n)
     $\vee$  (|AM.Usuarios|  $\leq$  n  $\wedge_L$  |res| = |AM.Usuarios|)
     $\wedge_L$  perteneceUsuarios(AM.Usuarios, res)
     $\wedge_L$  sinRepetidos(res)
     $\wedge_L$  ordenDecreciente(AM.Usuarios, res)
     $\wedge_L$  losMejoresCandidatos(AM.Usuarios, AM.Historial, res)
  }
}

```

```

proc ResumenTransacciones(in AM : Airmillas, in id :  $\mathbb{Z}$ ) : (< $\mathbb{R}$ ,  $\mathbb{R}$ ,  $\mathbb{R}$ > {
  requiere {
    id  $\in$  claves(AM.Usuarios)
  }
  asegura {
    res0 = MillasObtenidasPorVuelo(AM.Transacciones, id)  $\wedge_L$ 
    res1 = MillasDescontadasPorCanjeo(AM.Transacciones, id)  $\wedge_L$ 
    res2 = MillasTransferidas(AM.Transacciones, id, id)
  }
}

```

```

proc IniciarPromocion(inout AM : Airmiles, in nombrePromo : seq<char>, in factorPromo :
 $\mathbb{R}$ ,) {
  Discutir el tercer elemento de la lista [factorPromo,0,0]. NoPromo esta siempre en el sistema, no se puede pisar. Discutir si seria != noPromo o si directamente no tiene que pertenecer (podria existir y pisarse el valor?)
  requiere {

```

```

     $AM = AM0 \wedge factorPromo \geq 1 \wedge nombrePromo \notin claves(AM.Promociones)$ 
  }
  asegura {
     $AM.Usuarios = AM0.Usuarios \wedge$ 
     $AM.Historial = AM0.Historial \wedge$ 
     $AM.Promociones = SetKey(AM0.Promociones, nombrePromo, \langle factorPromo, 0, -1 \rangle)$ 
  }
}

```

```

proc IniciarPromocionConTope(inout  $AM : \text{Airmiles}$ , in  $nombrePromo : \text{string}$ , in  $factorPromo : \mathbb{R}$ , in  $TopePromo : \mathbb{R}$ ) {
  requiere {
     $AM = AM0 \wedge factorPromo \geq 1 \wedge nombrePromo \notin claves(AM.Promociones) \wedge$ 
     $TopePromo > 0$ 
  }
  asegura {
     $AM.Usuarios = AM0.Usuarios \wedge$ 
     $AM.Historial = AM0.Historial \wedge$ 
     $AM.Promociones = SetKey(AM0.Promociones, nombrePromo,$ 
     $\langle factorPromo, topePromo, topePromo \rangle)$ 
  }
}

```

```

proc FinalizarPromocion(inout  $AM : \text{Airmiles}$ , in  $nombrePromo : \text{seq}(\text{char})$ ) {
  requiere {
     $AM = AM0 \wedge nombrePromo \neq "NoPromo" \wedge_L$ 
     $nombrePromo \in claves(AM0.promociones)$ 
  }
  asegura {
     $AM.Usuarios = AM0.Usuarios \wedge$ 
     $AM.Historial = AM0.Historial \wedge$ 
     $AM.Promociones = DelKey(AM0.Promociones, nombrePromo)$ 
  }
}

```

```

proc ElQueTieneMasMillas(in  $AM : \text{Airmiles}$ , ) :  $\mathbb{Z}$  {
  requiere {  $|AM.Usuarios| > 0$  }
  asegura {

```

$$\left. \begin{array}{l} (\exists i : \mathbb{Z}) (i \in \text{claves}(AM.U\text{suarios}) \wedge_L \text{MillasTotales}(AM.H\text{istorial}, i) = \text{res}) \wedge \\ (\forall j : \mathbb{Z}) (j \in \text{claves}(AM.U\text{suarios}) \rightarrow_L \text{MillasTotales}(AM.H\text{istorial}, j) \leq \text{res}) \\ \text{Devuelve millas y deberia devolver un usuario?} \end{array} \right\}$$

```

proc ElMasCanjeador(in AM : Airmiles) :  $\mathbb{Z}$  {
  requiere {  $|AM.Usuarios| > 0 \wedge |AM.Historial| > 0$  }
  asegura {
     $(\exists id : \mathbb{Z}) (id \in claves(AM.Usuarios) \wedge_L res = id$ 
     $\wedge (\forall id2 : \mathbb{Z}) (id2 \in claves(AM.Usuarios) \rightarrow_L$ 
     $MillasPerdidas(AM.Historial, res, AM.Usuarios) \geq$ 
     $MillasPerdidas(AM.Historial, id2, AM.Usuarios)))$ 
  }
}

```

```

proc paresTransferidoresExclusivos(in Am : Airmiles) : seq<<Z,Z>> {
  requiere { True }
  asegura {
    NoHayRepetidos(res) ∧
    NoHayPermutacion(res) ∧
    TieneReciprocoHistorial(AM.Historial, res) ∧
    SonParejaExclusiva(AM.Historial, res)
  }
}

pred perteneceUsuarios(usuarios : Diccionario<Z, categoria>, res : seq<Z>) {
  (∀i : Z) (0 ≤ i < |res| →L res[i] ∈ claves(usuarios))
}

pred sinRepetidos(res : seq<Z>) {
  (∀x : Z) (x ∈ res →L cantAparicion(res, x) = 1)
}

aux cantAparicion(res : seq<Z>, x : Z) : ℝ =
  ∑i=0|res|-1 IfThenElse(res[i] = x, 1, 0)

pred ordenDecreciente(Historial : seq<Transacciones>, res : seq<Z>) {
  (∀i : Z) (0 ≤ i < |res| - 1 →L
    millasDisponibles(res[i], Historial) ≥ millasDisponibles(res[i + 1], Historial))
}

pred losMejoresCandidato(usuarios : Diccionario<Z, categoria>,
  Historial : seq<Transacciones>, res : seq<Z>) {
  (∀i : Z) (0 ≤ i < |res| ∧ res[i] ∈ claves(usuarios) →L
    elMejorCandidato(Usuarios, Historial, res, i))
}

pred elMejorCandidato(usuarios : Diccionario<Z, categoria>,
  Historial : seq<Transacciones>, res : seq<Z>, i : Z) {

```

```

    ( $\forall x : \mathbb{Z}$ ) ( $(x \in \text{claves}(\text{usuarios}) \wedge$ 
     $\text{millasDisponibles}(x, \text{Historial}) \leq \text{millasDisponibles}(\text{res}[i], \text{Historial})) \rightarrow_L$ 
     $(\exists j : \mathbb{Z}) (0 \leq j < i) \wedge_L \text{res}[j] = x)$ )
  }

```

```

pred SubioCategoria( $\text{idUusuario} : \mathbb{Z}, \text{Historial} : \text{Transaccion}, \text{Historial0} : \text{Transaccion}$ )
{
  DarCategoria( $\text{Historial0}, \text{idUusuario}$ )  $\neq$  DarCategoria( $\text{Historial}, \text{idUusuario}$ )
}

```

```

aux MillasCalculadas( $\text{distancia} : \mathbb{Z}, \text{Factorcategoria} : \mathbb{R}, \text{factorPromo} : \mathbb{R}$ ) :  $\mathbb{R} =$ 
   $\text{distancia} * \text{Factorcategoria} * \text{factorPromo}$ 

```

Con respecto a las transacciones, deberiamos limitar esto no? cuando el usuario es platino creo que no tendria sentido crearle transferencias para que gane millas, se supone que no podria. Por el contrario si puede crear transacciones que de millas

```

pred EsNuevaTransaccion( $\text{AM}, \text{AM0} : \text{AirMiles}, \text{idUusuario} : \mathbb{Z}, \text{idReferidor} : \mathbb{Z}, \text{TipoDeTransaccion} : \mathbb{Z},$ 
 $\text{Millas} : \mathbb{R}, \text{distancia} : \mathbb{Z}, \text{NombrePromo} : \text{string}, \text{FactorPromo} : \mathbb{R}$ ) {
   $\text{AM.Historial} = \text{AM0.Historial} + +[\text{idUusuario}, \text{idReferidor},$ 
   $\text{TipoDeTransaccion}, \text{Millas}, \text{NombrePromo}, \text{FactorPromo}]$ 
}

```

```

aux DarFactorCategoria( $\text{Historial} : \text{seq}\langle \text{Transacciones} \rangle, \text{idUusuario} : \mathbb{Z}$ ) :  $\mathbb{R} =$ 
  IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 10000, 1.0$ 
  , IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 50000, 1.5$ 
  , IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 1000000, 2.0, 0.0$ )))

```

```

aux DarCategoria( $\text{Historial} : \text{seq}\langle \text{Transacciones} \rangle, \text{idUusuario} : \mathbb{Z}$ ) : Categoria =
  IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 10000, \text{Bronce}$ 
  , IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 50000, \text{Plata}$ 
  , IfThenElse( $\text{MillasTotales}(\text{Historial}, \text{idUusuario}) < 1000000, \text{Oro}, \text{Platino}$ )))

```

En caso de que el usuario haga parte del viaje con promo y otra parte sin promo esta funcion busca esa diferencia

```

aux MillasACompletar( $\text{Promociones} : \text{Diccionario}\langle \text{string}, \text{struct} \rangle,$ 
 $\text{nombrePromo} : \text{string}, \text{distancia} : \mathbb{Z}, \text{FactorCategoria} : \mathbb{R}$ ) :  $\mathbb{R} =$ 
   $\text{MillasCalculadas}(\text{distancia}, \text{factorCategoria},$ 
   $\text{Promociones}[\text{nombrePromo}].\text{factorDePromocion}) -$ 
   $\text{Promociones}[\text{nombrePromo}].\text{millasRestantes}$ 

```

¿puedo chequear como va la sumatoria mientras va recorriendo?

```

aux MillasTotales( $\text{IdUusuario} : \mathbb{Z}, \text{Historial} : \text{seq}\langle \text{Transacciones} \rangle$ ) :  $\mathbb{Z} =$ 
   $\text{res} = \sum_{i=0}^{|\text{Historial}|-1} \text{IfThenElse}(\text{EsTransaccionDelUusuario}(\text{Historial}[i], \text{IdUusuario} \wedge$ 

```

$EsTransaccionSumadora(Historial[i], IdU\ suario))$
 $\wedge 1000000 > res, Historial[i].Millas, 0)$

Auxiliar para obtener las millas disponibles de un usuario. Recorre el historial de transacciones y de acuerdo al TipoDeTransaccion, al IdUsuario y las Millas del registro, suma o resta con un acumulador.

aux MillasDisponibles($IdU\ suario : \mathbb{Z}, Historial : seq\langle Transacciones \rangle$) : $\mathbb{Z} =$
 $res = \sum_{i=0}^{|Historial|-1} \text{IfThenElse}(EsTransaccionDelUsuario(Historial[i], IdU\ suario),$
 $\text{IfThenElse}(EsTransaccionSumadora(Historial[i], IdU\ suario)$
 $, Historial[i].Millas, -Historial[i].Millas), 0)$

pred EsTransaccionSumadora($Trans : Transacciones, idU\ suario : \mathbb{Z}$) {
 $(Trans.TipoDeTransaccion = 0) \vee$
 $(IdU\ suario = Trans.UsuarioDestino \wedge Trans.TipoDeTransaccion = 1)$
 $}$

pred EsTransaccionDelUsuario($Trans : Transacciones, idU\ suario : \mathbb{Z}$) {
 $Trans.UsuarioOrigen = idU\ suario \vee Trans.UsuarioDestino = idU\ suario$ }
Valido que el IdUsuario este en alguno de los 2 campos. Si no esta en ninguno, entonces no debo sumar ni restar nada.

pred SeConsumioLaPromocion($nombrePromo : string, AM : AirmilesAM0 : Airmiles$)
{
 $AM.Promociones = DelKey(AM0.Promociones, nombrePromo)$
 $}$

aux MillasObtenidasPorVuelo($Historial : seq\langle Transacciones \rangle, idOrigen : \mathbb{Z}$) : $\mathbb{R} =$
 $\sum_{i=0}^{|Historial|-1} \text{IfThenElse}(EsVuelo(Historial[i], idOrigen), Historial[i].Millas, 0)$

aux MillasTransferidas($Historial : seq\langle Transacciones \rangle, idOrigen : \mathbb{Z}, idDestino : \mathbb{Z}$) :
 $\mathbb{R} =$
 $\sum_{i=0}^{|Historial|-1} \text{IfThenElse}(EsTransferencia(Historial[i],$
 $idOrigen, idDestino), Historial[i].Millas, 0)$

aux MillasDescontadasPorCanjeo($Historial : seq\langle Transacciones \rangle, idOrigen : \mathbb{Z}$) : $\mathbb{R} =$
 $\sum_{i=0}^{|Historial|-1} \text{IfThenElse}(EsCanjeo(Historial[i], idOrigen), Historial[i].Millas, 0)$

pred EsVuelo($Transaccion : Struct, idOrigen : \mathbb{Z}$) {

```

    Transaccion.UsuarioOrigen = idOrigen ∧
    Transaccion.TipoDeTransaccion = 0 ∧
    Transaccion.UsuarioDestino = -1 ∧
    Deberia verificar los otros campos? Se complica...
}

```

```

pred EsTransferencia(Transaccion : struct, idOrigen : ℤ, idDestino : ℤ) {
    Transaccion.UsuarioOrigen = idOrigen ∧
    Transaccion.TipoDeTransaccion = 1 ∧
    Transaccion.UsuarioDestino = idDestino ∧
    Transaccion.millas > 0
    Deberia verificar los otros campos? Se complica...
}

```

```

pred EsCanjeo(Transaccion : struct, idOrigen : ℤ) {
    Transaccion.UsuarioOrigen = idOrigen ∧
    Transaccion.TipoDeTransaccion = 2 ∧
    Transaccion.UsuarioDestino = -1 ∧
    Transaccion.distancia > 0
    Deberia verificar los otros campos? Se complica...
}

```

Es similar a EsTransferencia pero aca no importa el destino, unicamente chequeo que exista el usuario de origen.

```

pred EsTransferenciaEnviada(Transaccion : Transacciones, idOrigen : ℤ, Usuarios :
dict⟨ℤ, Categoria⟩) {
    Transaccion.UsuarioOrigen = idOrigen ∧
    Transaccion.TipoDeTransaccion = 1 ∧
    Transaccion.UsuarioDestino ∈ claves(Usuarios) ∧
    Transaccion.UsuarioDestino ≠ idOrigen
    Deberia verificar los otros campos? Se complica...
}

```

```

aux MillasPerdidas(Historial : seq⟨Transacciones⟩, idUsuario : ℤ, Usuarios : Diccionario⟨ℤ, categoria⟩) :
ℝ =

```

```

    |Historial|-1
    ∑i=0 IfThenElse(EsTransferenciaEnviada(Historial[i], idUsuario, Usuarios) ∨

```

```

        EsCanjeo(Historial[i], idUsuario), Historial[i].Millas, 0)

```

```

pred NoHayRepetidos(res : seq⟨⟨ℤ, ℤ⟩⟩) {
    (∀t : ⟨ℤ, ℤ⟩) (t ∈ res →L cantApariciones(t, res) = 1)
}

```

```

aux cantApariciones(t : ⟨ℤ, ℤ⟩, res : seq⟨ℤ, ℤ⟩) : ℤ =
    ∑0|res| IfThenElse(t ∈ res, 1, 0)

```

Cual es la forma correcta de hacer esto? algo asi pero tengo que indicar el orden en el que estoy poniendo las cosas?

```

pred NoHayPermutacion(res) {
  (∀t : ⟨ℤ, ℤ⟩) (t ∈ res →L ⟨t1, t0⟩ ∉ res)
}

pred TieneReciprocoHistorial(Historial : seq⟨Transacciones⟩, res : seq⟨⟨ℤ, ℤ⟩⟩) {
  (∀x : ⟨ℤ, ℤ⟩) (x ∈ res → HayReciproco(Historial, res))
}

pred HayReciproco(Historial : seq⟨Transacciones⟩, t : ⟨ℤ, ℤ⟩) {
  (∃i : ℤ) (0 ≤ i < |Historial| ∧L Historial[i].TipoDeTransaccion = 1 ∧L
    (Historial[i].usuarioOrigen = t1 ∧ Historial[i].usuarioDestino = t0))
}

Revisar
pred SonParejaExclusiva(Historial : seq⟨Transacciones⟩, res : seq⟨⟨ℤ, ℤ⟩⟩) {
  (∀t : ⟨ℤ, ℤ⟩) (t ∈ res →L ¬(hayOtroUsuario(t0, t1, Historial) ∨
    hayOtroUsuario(t1, t0, Historial)))
  hace falta dos veces hay otro usuario?
  si hay otro usuario en uno de los dos, me devuelve true
}

pred HayOtroUsuario(t0, t1, Historial) {
  (∀i : ℤ) (0 ≤ i < |Historial| →L (Historial[i].UsuarioOrigen = t0 ∧
    Historial[i].UsuarioDestino ≠ t1))
  Esto es siempre False o casi siempre, porque el usuario origen en cada transaccion no es t0, deberi
}

```